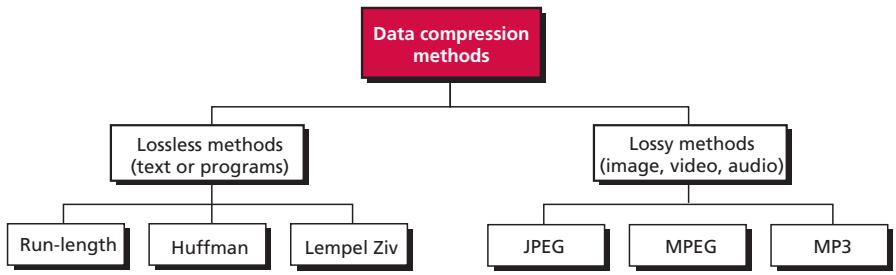
15.1 INTRODUCTION

Data compression implies sending or storing a smaller number of bits. Although many methods are used for this purpose, in general these methods can be divided into two broad categories: lossless and lossy methods. Figure 15.1 shows the two categories and common methods used in each category.

We first discuss lossless compression methods, as they are simpler and easier to understand. We then present lossy compression methods.

Figure 15.1 *Data compression methods*



15.2 LOSSLESS COMPRESSION METHODS

In **lossless data compression**, the integrity of the data is preserved. The original data and the data after compression and decompression are exactly the same because, in these methods, the compression and decompression algorithms are exact inverses of each other: no part of the data is lost in the process. Redundant data is removed in compression and added during decompression.

Lossless compression methods are normally used when we cannot afford to lose any data. For example, we must not lose data when we compress a text file or an application program.

We discuss three lossless compression methods in this section: *run-length encoding*, *Huffman coding*, and the *Lempel Ziv algorithm*.

15.2.1 Run-length encoding

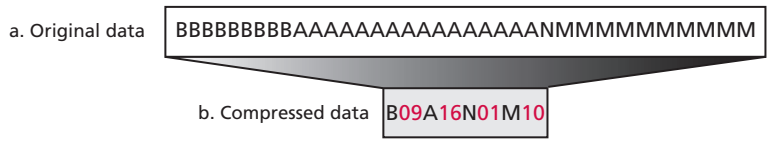
Run-length encoding is probably the simplest method of compression. It can be used to compress data made of any combination of symbols. It does not need to know the frequency of occurrence of symbols (as is necessary for Huffman coding) and can be very efficient if data is represented as 0s and 1s.

The general idea behind this method is to replace consecutive repeating occurrences of a symbol by one occurrence of the symbol followed by the number of occurrences. For example, AAAAAAAA can be replaced by A08. Figure 15.2 shows an example of this simple compression method. Note that we use a fixed number of digits (two) to represent the count.

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

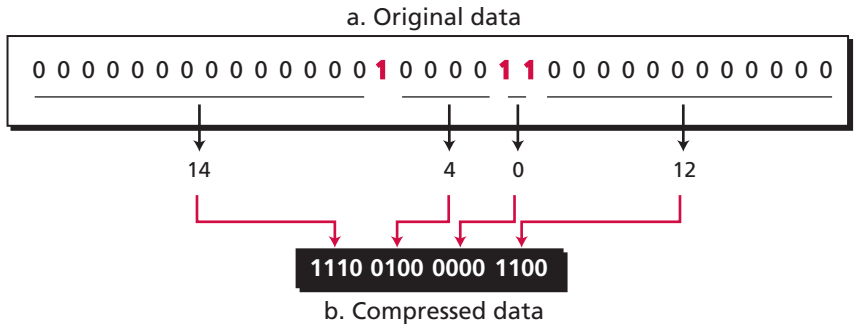
Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

Figure 15.2 *Run-length encoding example*



The method can be even more efficient if the data uses only two symbols (for example 0 and 1) in its bit pattern and one symbol is more frequent than the other. For example, let's say we have an image represented by mostly 0s and some 1s. In this case, we can reduce the number of bits by sending (or storing) the number of 0s occurring between two 1s (Figure 15.3).

Figure 15.3 *Run-length encoding for two symbols*



We have represented the counts as a 4-bit binary number (unsigned integer). In an actual situation, we would find an optimal number of bits to avoid introducing extra redundancy. In Figure 15.3, there are fourteen 0s before the first 1. These fourteen 0s are compressed to the binary pattern 1110 (14 in binary). The next set of 0s is compressed to 0100 because there are four 0s. Next we have two 1s in the original data, which are represented by 0000 in the compressed data. Finally, the last twelve 0s in the data are compressed to 1100.

Note that, given a 4-bit binary compression, if there are more than fifteen 0s, they are broken into two or more groups. For example, a sequence of twenty-five 0s is encoded as 1111 1010. Now the question is how the decoding algorithm knows that this consists of twenty-five 0s and not fifteen 0s, then a 1, and then ten 0s. The answer is that if the first count is 1111, the receiver knows the next 4-bit pattern is a continuation of 0s. Now another question is raised: what if there are exactly fifteen 0s between two 1s? In this case, the pattern is 1111 followed by 0000.

15.2.2 Huffman coding

Huffman coding assigns shorter codes to symbols that occur more frequently and longer codes to those that occur less frequently. For example, imagine we have a text file that uses only five characters (A, B, C, D, E). We chose only five characters to make the discussion simpler, but the procedure is equally valid for a smaller or greater number of characters.

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

Before we can assign bit patterns to each character, we assign each character a weight based on its frequency of use. In this example, assume that the frequency of the characters is as shown in Table 15.1. Character A occurs 17 per cent of the time, character B occurs 12 per cent of the time, and so on.

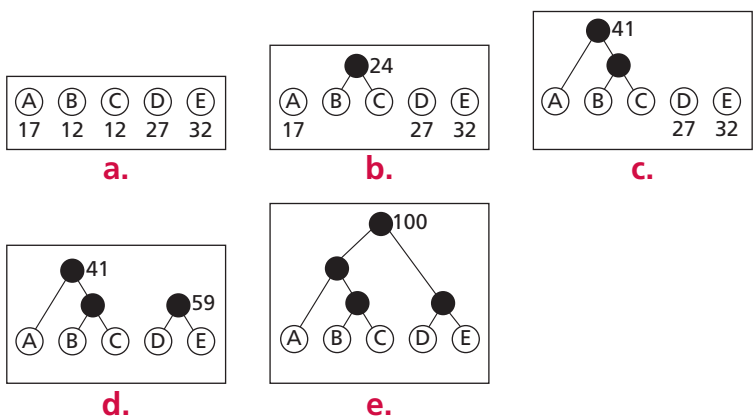
Table 15.1 Frequency of characters

Character	A	B	C	D	E
Frequency	17	12	12	27	32

Once the weight of each character is established, we build a tree based on those values. The process for building this tree is shown in Figure 15.4. It follows three basic steps:

1. Put the entire character set in a row. Each character is now a node at the lowest level of the tree.
2. Find the two nodes with the smallest weights and join them to form a third node, resulting in a simple two-level tree. The weight of the new node is the combined weights of the original two nodes. This node, one level up from the leaves, is eligible for combination with other nodes. Remember that the sum of the weights of the two nodes chosen must be smaller than the combination of any other possible choices.
3. Repeat step 2 until all of the nodes, on every level, are combined into a single tree.

Figure 15.4 *Huffman coding*



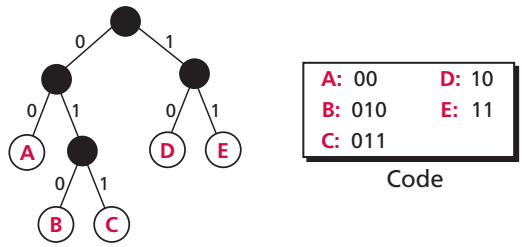
Once the tree is complete, use it to assign codes to each character. First, assign a bit value to each branch. Starting from the root (top node), assign 0 to the left branch and 1 to the right branch and repeat this pattern at each node.

A character's code is found by starting at the root and following the branches that lead to that character. The code itself is the bit value of each branch on the path, taken in sequence. Figure 15.5 shows the final tree with bits added to each branch. Note that we moved the leaf nodes to make the tree look like a binary tree.

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

Figure 15.5 *Final tree and code*

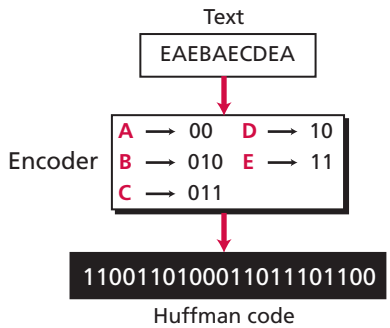


Note these points about the codes. First, the characters with higher frequencies receive a shorter code (A, D, and E) than the characters with lower frequencies (B and C). Compare this with a code that assigns equal bit lengths to each character. Second, in this coding system, no code is a prefix of another code. The 2-bit codes, 00, 10, and 11, are not the prefixes of any of the two other codes (010 and 011). In other words, we do not have a 3-bit code beginning with 00, 10, or 11. This property makes Huffman code an instantaneous code. We will explain this property when we discuss encoding and decoding in Huffman coding.

Encoding

Let us see how to encode text using the code for our five characters. Figure 15.6 shows the original and the encoded text.

Figure 15.6 *Huffman encoding*



Two points about this figure are worth mentioning. First, notice that there is a sense of compression even in this small and unrealistic code. If we want to send the text without using Huffman coding, we need to assign a 3-bit code to each character. You would have sent 30 bits, whereas with Huffman coding, we send only 22 bits.

Second, notice that we have not used any delimiters between the bits that encode each character. We write the codes one after another. The beauty of Huffman coding is that no code is the prefix of another code. There is therefore no ambiguity in encoding, so the decoding algorithm can decode the received data without ambiguity.

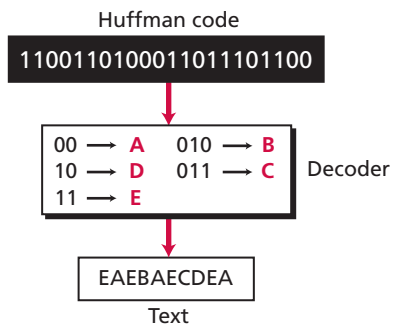
Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

Decoding

The recipient has a very easy job in decoding the data that it receives. Figure 15.7 shows how decoding takes place. When the recipient receives the first 2 bits, it does not have to wait for the next bit to make a decision—it knows that these 2 bits encode the letter E. This is because these 2 bits are not the prefix of any 3-bit code (there is no 3-bit code that starts with 11). Likewise, when the receiver receives the next 2 bits (00), it also knows that the character must be A. The next 2 bits are interpreted the same way (11 must be E). However, when it receives bits 7 and 8, it knows that it must wait for the next bit, because this code (01) is not in the list of codes. After receiving the next bit (0), it interprets the 3 bits together (010) as B. This is why Huffman code is called an instantaneous code—the decoder can unambiguously decode the bits instantaneously, using the minimum number of bits.

Figure 15.7 *Huffman decoding*



15.2.3 Lempel Ziv encoding

Lempel Ziv (LZ) encoding, named after its inventors (Abraham Lempel and Jacob Ziv), is an example of a category of algorithms called **dictionary-based encoding**. The idea is to create a dictionary (a table) of strings used during the communication session. If both the sender and the receiver have a copy of the dictionary, then previously encountered strings can be substituted by their index in the dictionary to reduce the amount of information transmitted.

Although the idea appears simple, several difficulties surface in the implementation. First, how can a dictionary be created for each session? It cannot be universal, due to its length. Second, how can the recipient acquire the dictionary created by the sender—if we send the dictionary, we are sending extra data, which defeats the whole purpose of compression?

The Lempel Ziv (LZ) algorithm is a practical algorithm that uses the idea of adaptive dictionary-based encoding. The algorithm has gone through several versions (LZ77, LZ78). We introduce the basic idea of this algorithm with an example, but do not delve into the details of different versions and implementations. In the example, assume that the following string is to be sent. We have chosen this specific string to simplify the discussion:

BAABABBBAABBBBAA

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

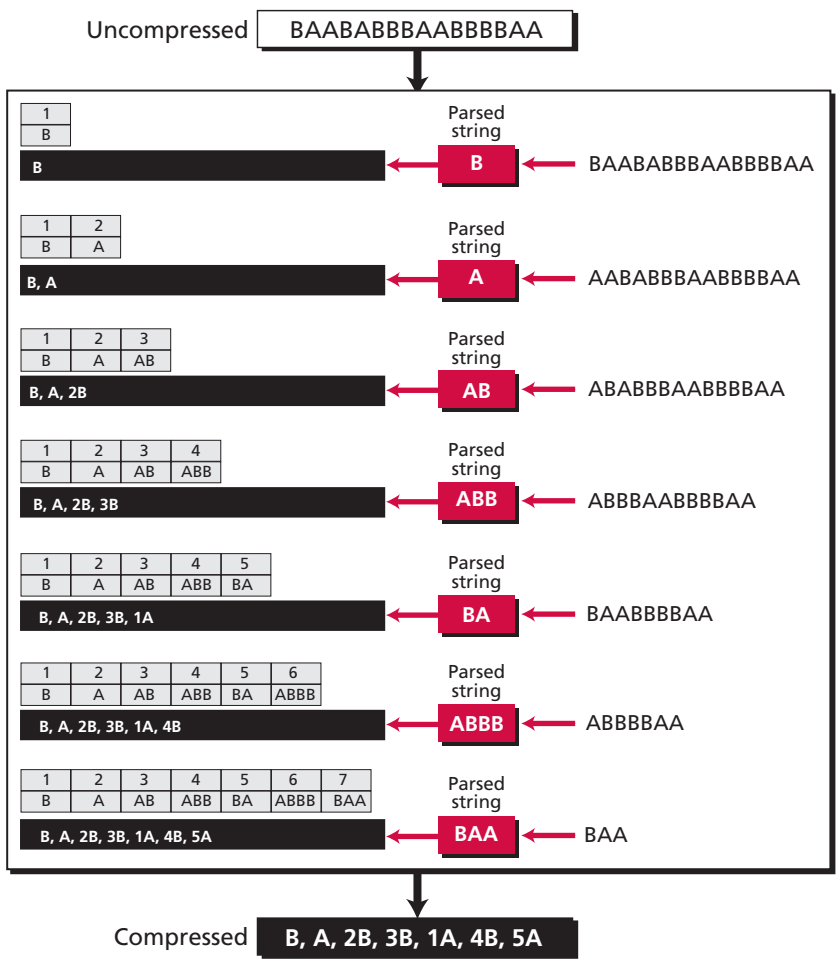
Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

Using our simple version of the LZ algorithm, the process is divided into two phases: compressing the string and decompressing the string.

Compression

In this phase there are two concurrent events: building an indexed dictionary and compressing a string of symbols. The algorithm extracts the smallest substring that cannot be found in the dictionary from the remaining uncompressed string. It then stores a copy of this substring in the dictionary as a new entry and assigns it an index value. Compression occurs when the substring, except for the last character, is replaced with the index found in the dictionary. The process then inserts the index and the last character of the substring into the compressed string. For example, if the substring is ABBB, we search for ABB in the dictionary. You find that the index for ABB is 4, so the compressed substring is therefore 4B. Figure 15.8 shows the process for our sample string.

Figure 15.8 *An example of Lempel Ziv encoding*



Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

Let us go through a few steps in this figure:

Step 1

The encoding process extracts the smallest substring from the original string that is not in the dictionary. Because the dictionary is empty, the smallest character is one character (the first character, B). The process stores a copy of it as the first entry in the dictionary with an index of 1. No part of this substring can be replaced with an index from the dictionary, as it is only one character. The process inserts B in the compressed string. So far, the compressed string has only one character, B. The remaining uncompressed string is the original string without the first character.

Step 2

The encoding process extracts the next smallest substring that is not in the dictionary from the remaining string. This substring is the character A, which is not in the dictionary. The process stores a copy of it as the second entry in the dictionary. No part of this substring can be replaced with an index from the dictionary, as it is only one character. The process inserts A in the compressed string. So far, the compressed string has two characters: B and A (we have placed commas between the substrings in the compressed string to show the separation).

Step 3

The encoding process extracts the next smallest substring that is not in the dictionary from the remaining string. This situation differs from the two previous steps: the next character (A) **is** in the dictionary, so the process extracts two characters (AB), which are not in the dictionary. The process stores a copy of AB as the third entry in the dictionary. The process now finds the index of an entry in the dictionary that is the substring without the last character (AB without the last character is A). The index for A is 2, so the process replaces A with 2 and inserts 2B in the compressed string.

Step 4

Next the encoding process extracts the substring ABB (because A and AB are already in the dictionary). A copy of ABB is stored in the dictionary with an index of 4. The process finds the index of the substring without the last character (AB), which is 3. The combination 3B is inserted into the compressed string.

You may have noticed that in the three previous steps, we have not actually achieved any compression, because we have replaced one character by one character (A by A in the first step and B by B in the second step) and two characters by two characters (AB by 2B in the third step). But in this step, we have reduced the number of characters (ABB becomes 3B). If the original string has many repetitions (which is true in most cases), we can greatly reduce the number of characters.

The remaining steps are all similar to one of the preceding four steps. Note that the dictionary was only used by the encoding program to find the indexes. It is not sent to the recipient, and the recipient must create the dictionary for itself, as we will see in the next section.

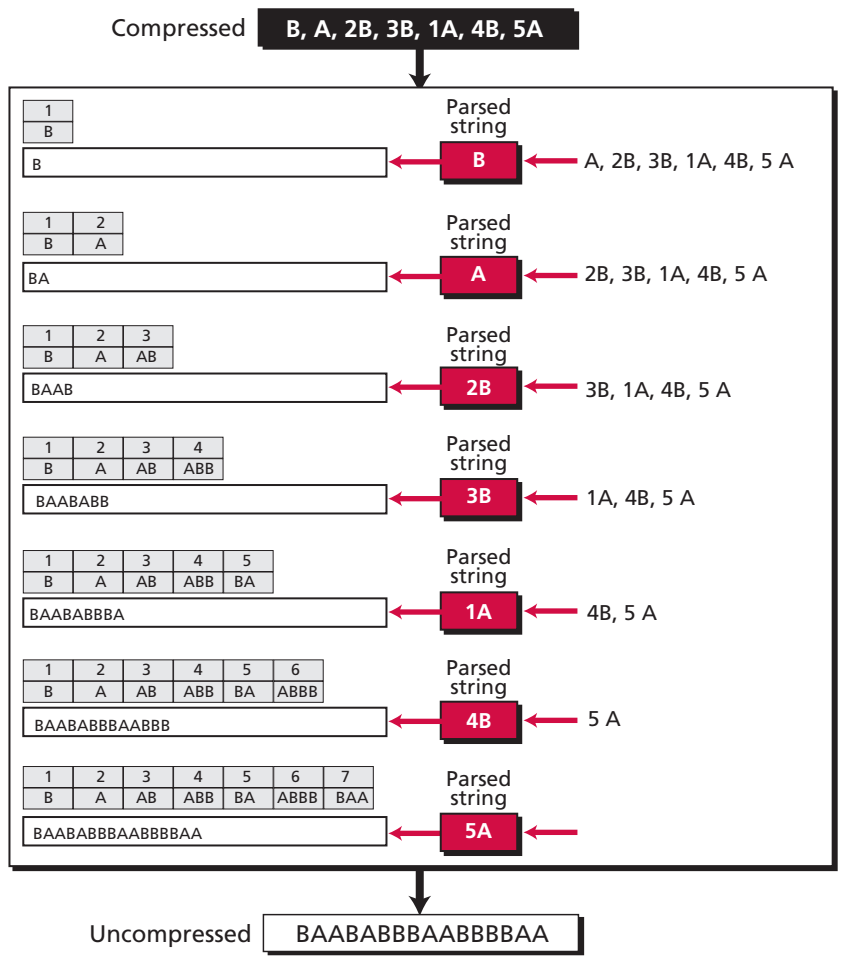
Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

Decompression

Decompression is the inverse of the compression process. The process extracts the substrings from the compressed string and tries to replace the indexes with the corresponding entry in the dictionary, which is empty at first and built up gradually. The idea is that when an index is received, there is already an entry in the dictionary corresponding to that index. Figure 15.9 shows the decompression process.

Figure 15.9 *An example of Lempel Ziv decoding*



Let us go through a few steps in the figure:

Step 1

The first substring of the compressed string is examined. It is B without an index. Because the substring is not in the dictionary, it is added to the dictionary. The substring (B) is inserted into the decompressed string.

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

Step 2

The second substring (A) is examined: the situation is similar to step 1. Now the decompressed string has two characters (BA), and the dictionary has two entries.

Step 3

The third substring (2B) is examined. The process searches the dictionary and replaces the index 2 with the substring A. The new substring (AB) is added to the decompressed string, and AB is added to the dictionary.

Step 4

The fourth substring (3B) is examined. The process searches the dictionary and replaces the index 3 with the substring AB. The substring ABB is now added to the decompressed string, and ABB is added to the dictionary.

We leave the exploration of the last three steps as an exercise. As we have noticed, we used a number such as 1 or 2 for the index. In reality, the index is a binary pattern (possibly variable in length) for better efficiency. Also note that LZ encoding leaves the last character uncompressed (which means less efficiency). A version of LZ encoding, called **Lempel Ziv Welch (LZW) encoding**, compresses even this single character. However, we leave the discussion of this algorithm to more specialized textbooks.

15.3 LOSSY COMPRESSION METHODS

Loss of information is not acceptable in a text file or a program file. It is, however, acceptable in an image, video, or audio file. The reason is that our eyes and ears cannot distinguish subtle changes. In such cases, we can use a **lossy data compression** method. These methods are cheaper—they take less time and space when it comes to sending millions of bits per second for images and video.

Several methods have been developed using lossy compression techniques. **JPEG (Joint Photographic Experts Group)** encoding is used to compress pictures and graphics, **MPEG (Moving Picture Experts Group)** encoding is used to compress video, and **MP3 (MPEG audio layer 3)** for audio compression.

15.3.1 Image compression—JPEG encoding

As discussed in Chapter 2, an image can be represented by a two-dimensional array (table) of picture elements (pixels). For example, $640 \times 480 = 307\,200$ pixels. If the picture is grayscale, each pixel can be represented by an 8-bit integer, giving 256 levels of gray. If the picture is color, each pixel can be represented by 24 bits (3×8 bits), with each 8 bits representing one of the colors in the RBG color system. To simplify the discussion, we concentrate on a grayscale picture with 640×480 pixels. You can see why we need compression. A grayscale picture of 307 200 pixels is represented by 2 457 600 bits, and a color picture is represented by 7 372 800 bits.

In JPEG, a grayscale picture is divided into blocks of 8×8 pixel blocks (Figure 15.10). The purpose of dividing the picture into blocks is to decrease the number of calculations because, as we will see shortly, the number of mathematical operations for each picture is the square of the number of units. That is, for the entire image, we need $307\,200^2$ operations

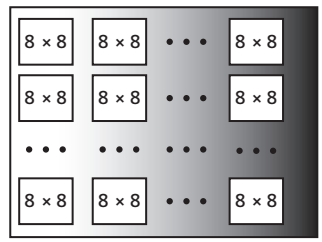
分块8x8目的：减少计算量

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

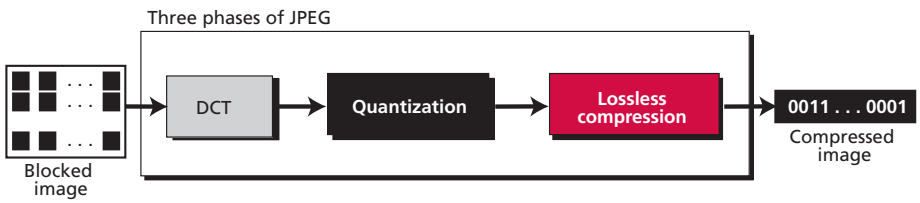
(94 371 840 000 operations). If we use JPEG, we need 64^2 operations for each block, a total of $64^2 \times 80 \times 60$, or 19 660 800 operations. This decreases by 4800 times the number of operations.

Figure 15.10 *JPEG grayscale example, 640 × 480 pixels*



The whole idea of JPEG is to change the picture into a linear (vector) set of numbers that reveals the redundancies. The redundancies (lack of changes) can then be removed using one of the lossless compression methods we studied previously. A simplified version of the process is shown in Figure 15.11.

Figure 15.11 *The JPEG compression process*



Discrete cosine transform (DCT)

In this step, each block of 64 pixels goes through a transformation called the **discrete cosine transform (DCT)**. The transformation changes the 64 values so that the relative relationships between pixels are kept but the redundancies are revealed. The formula is given in Appendix G. $P(x, y)$ defines one value in the block, while $T(m, n)$ defines the value in the transformed block.

To understand the nature of this transformation, let us show the result of the transformations for three cases.

Case 1

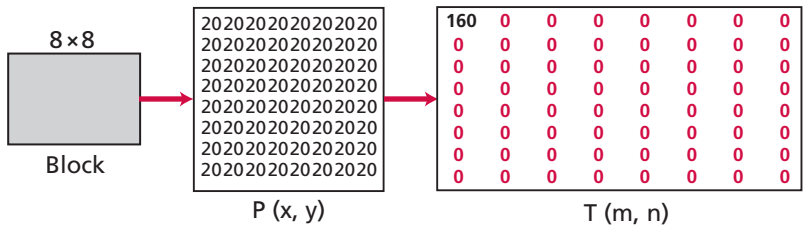
In this case, we have a block of uniform grayscale, and the value of each pixel is 20. When we do the transformations, we get a nonzero value for the first element (upper left corner). The rest of the pixels have a value of 0 because, according to the formula, the value

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

of $T(0,0)$ is the average of the other values. This is called the **DC value** (direct current, borrowed from electrical engineering). The rest of the values, called **AC values**, in $T(m, n)$ represent changes in the pixel values. But because there are no changes, the rest of the values are 0s (Figure 15.12).

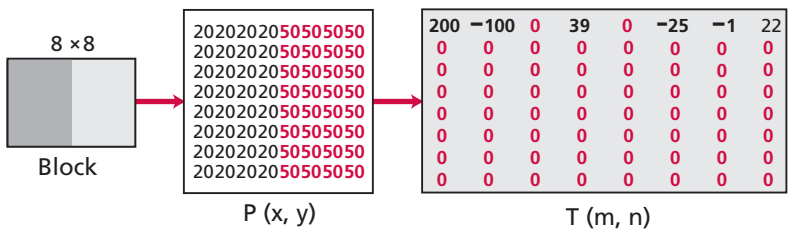
Figure 15.12 *Case 1: uniform grayscale*



Case 2

In the second case, we have a block with two different uniform grayscale sections. There is a sharp change in the values of the pixels (from 20 to 50). When we do the transformations, we get a DC value as well as nonzero AC values. However, there are only a few nonzero values clustered around the DC value. Most of the values are 0 (Figure 15.13).

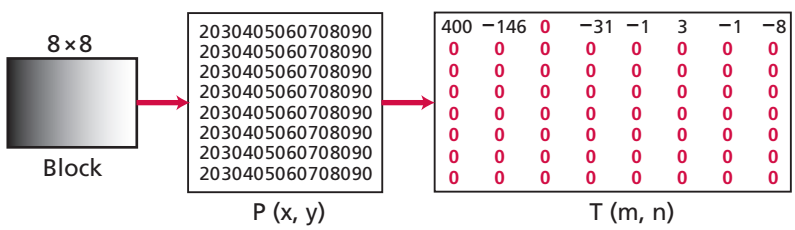
Figure 15.13 *Case 2: two sections*



Case 3

In the third case, we have a block that changes gradually. That is, there is no sharp change between the values of neighboring pixels. When we do the transformations, we get a DC value, with many nonzero AC values also (Figure 15.14).

Figure 15.14 *Case 3: gradient grayscale*



Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

From Figures 15.12, 15.13, and 15.14, we can state the following:

- The transformation creates table *T* from table *P*.
- The DC value gives the average value of the pixels.
- The AC values gives the changes.
- Lack of changes in adjacent pixels creates 0s.

Note that the DCT transformation is reversible. Appendix G also shows the mathematical formula for a reverse transformation.

Quantization

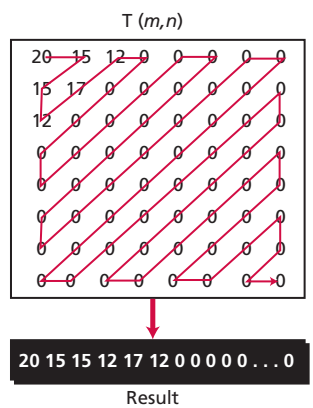
After the *T* table is created, the values are quantized to reduce the number of bits needed for encoding. Quantization divides the number of bits by a constant and then drops the fraction. This reduces the required number of bits even more. In most implementations, a quantizing table (8 by 8) defines how to quantize each value. The divisor depends on the position of the value in the *T* table. This is done to optimize the number of bits and the number of 0s for each particular application.

Note that the only phase in the process that is not reversible is the quantizing phase. You lose some information here that is not recoverable. The only reason that JPEG is a lossy compression method is because of the quantization phase.

Compression

After quantization the values are read from the table, and redundant 0s are removed. However, to cluster the 0s together, the process reads the table diagonally in a zigzag fashion rather than row by row or column by column. The reason is that if the picture does not have fine changes, the bottom right corner of the *T* table is all 0s. Figure 15.15 shows the process. JPEG usually uses run-length encoding at the compression phase to compress the bit pattern resulting from the zigzag linearization.

Figure 15.15 *Reading the table*



Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

15.3.2 Video compression—MPEG encoding

The Moving Picture Experts Group (MPEG) method is used to compress video. In principle, a motion picture is a rapid sequence of a set of frames in which each frame is a picture. In other words, a frame is a spatial combination of pixels, and a video is a temporal combination of frames that are sent one after another. Compressing video, then, means spatially compressing each frame and temporally compressing a set of frames.

Spatial compression

The **spatial compression** of each frame is done with JPEG, or a modification of it. Each frame is a picture that can be independently compressed.

Temporal compression

In **temporal compression**, redundant frames are removed. When we watch television, for example, we receive 30 frames per second. However, most of the consecutive frames are almost the same. For example, in a static scene in which someone is talking, most frames are the same except for the segment around the speaker's lips, which changes from one frame to the next.

A rough calculation points to the need for temporal compression for video. A 20:1 JPEG compression of one frame sends 368 640 bits per frame: at 30 frames per second, this is 11 059 200 bits per second. We need to reduce this number!

To temporally compress data, the MPEG method first divides frames into three categories: I-frames, P-frames, and B-frames.

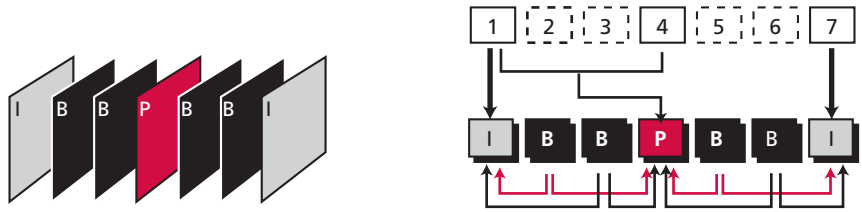
- **I-frames**. An **intracoded frame (I-frame)** is an independent frame that is not related to any other frame—that is, not to the frame sent before or to the frame sent after. They are present at regular intervals (for example, every ninth frame is an I-frame). An I-frame must appear periodically due to some sudden change in the frame that the previous and following frames cannot show. Also, when a video is broadcast, a viewer may tune in their receiver at any time. If there is only one I-frame at the beginning of the broadcast, the viewer who tunes in late will not receive a complete picture. I-frames are independent of other frames and cannot be constructed from other frames.
- **P-frames**. A **predicted frame ((P-frame)** is related to the preceding I-frame or P-frame. In other words, each P-frame contains only the changes from the preceding frame. The changes, however, cannot cover a big segment of the image. For example, for a fast-moving object, the new changes may not be recorded in a P-frame. P-frames can be constructed only from previous I- or P-frames. P-frames carry much less information than other frame types and carry even fewer bits after compression.
- **B-frames**. A **bidirectional frame (B-frame)** is relative to the preceding and following I-frame or P-frame. In other words, each B-frame is relative to the past and the future. Note that a B-frame is never related to another B-frame.

Figure 15.16 shows a sample sequence of frames and how they are constructed. Note that for decoding, the decoding process should receive the P frames before the B frames. For this reason, the order of transmission of frames is different than the order in which they displayed at the receiving application. The frames are sent as I, P, B, B, P, B, B, I.

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

Figure 15.16 *MPEG frames*



a. Frames b. Frame construction

Versions

MPEG has gone through several versions. The discussion above is related to MPEG-1. MPEG-2 was introduced in 1991, is more capable than MPEG and can be used for video storage as well as TV broadcasting, including high definition TV (HDTV). A more recent version of MPEG is called MPEG-7, which is named 'Multimedia Content Description Interface'. MPEG-7 is mostly a standard that uses XML to describe *metadata* (data about data), the description of what is included in the video.

15.3.3 Audio compression

Audio compression can be used for speech or music. For speech we need to compress a 64-kHz digitized signal, while for music we need to compress a 1.411-MHz signal. Two categories of techniques are used for audio compression: predictive encoding and perceptual encoding.

Predictive encoding

In **predictive encoding**, the differences between samples are encoded instead of encoding all the sampled values. This type of compression is normally used for speech. Several standards have been defined such as GSM (13 kbps), G.729 (8 kbps), and G.723.3 (6.4 or 5.3 kbps). Detailed discussions of these techniques are beyond the scope of this book.

Perceptual encoding: MP3

The most common compression technique used to create CD-quality audio is based on the **perceptual encoding** technique. This type of audio needs at least 1.411 Mbps, which cannot be sent over the Internet without compression. MP3 (MPEG audio layer 3), a part of the MPEG standard (discussed in the video compression section), uses this technique.

Perceptual encoding is based on the science of psychoacoustics, which is the study of how people perceive sound. The idea is based on flaws in our auditory system: some sounds can mask other sounds. Masking can happen in both frequency and time. In **frequency masking**, a loud sound in one frequency range can partially or totally mask a

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

softer sound in another frequency range. For example, we cannot hear what our dance partner says in a room in which a loud heavy metal band is performing. In **temporal masking**, a loud sound can reduce the sensitivity of our hearing for a short time even after the sound has stopped.

MP3 uses these two phenomena, frequency and temporal masking, to compress audio signals. The technique analyzes and divides the audio spectrum into several groups. Zero bits are allocated to frequency ranges that are totally masked, a small number of bits are allocated to frequency ranges that are partially masked, and a larger number of bits are allocated to frequency ranges that are not masked.

MP3 produces three data rates: 96 kbps, 128 kbps, and 160 kbps. The rate is based on the range of the frequencies in the original analog audio.

15.4 END-CHAPTER MATERIALS

15.4.1 Recommended reading

For more details about the subjects discussed in this chapter, the following books are recommended:

- Drozdek, A. *Elements of Data Compression*, Boston, MA: Course Technology, 2001
- Symes, P. *Video Compression*, New York: McGraw-Hill, 1998
- Haskell, B., Puri, A. and Netravali, A. *Digital Video: An Introduction to MPEG2*, New York: Chapman and Hall, 1997
- Pennebaker, W. and Mitchell, J. *JPEG Still Image Data Compression Standard*, New York: Van Nostrand Reinhold, 1993

15.4.2 Key terms

AC value 402	lossless data compression 392
bidirectional frame (B-frame) 404	lossy data compression 400
data compression 392	Moving Picture Experts Group (MPEG) 400
DC value 402	MPEG audio layer 3 (MP3) 400
dictionary-based encoding 396	perceptual encoding 405
discrete cosine transform (DCT) 401	predicted frame (P-frame) 404
frequency masking 405	predictive encoding 405
Huffman coding 393	run-length encoding 392
intracoded frame (I-frame) 404	spatial compression 404
Joint Photographic Experts Group (JPEG) 400	temporal compression 404
Lempel Ziv (LZ) encoding 396	temporal masking 406
Lempel Ziv Welch (LZW) encoding 400	

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.

15.4.3 Summary

- Data compression methods are either lossless (all information is recoverable) or lossy (some information is lost).
- In lossless compression methods, the received data is an exact replica of the sent data. Three lossless compression methods are run-length encoding, Huffman coding, and Lempel Ziv (LZ) encoding.
- In run-length encoding, repeated occurrences of a symbol are replaced by a symbol and the number of occurrences of the symbol.
- In Huffman coding, the code length is a function of symbol frequency: more frequent symbols have shorter codes than less frequent symbols.
- In LZ encoding, repeated strings or words are stored in memory locations. An index to the memory location replaces the string or word. LZ encoding requires a dictionary and an algorithm at both sender and receiver.
- In lossy compression methods, the received data need not be an exact replica of the sent data. Three lossy compression methods were discussed in this chapter: JPEG, MPEG, and MP3.
- JPEG (Joint Photographic Experts Group) compression is a method of compressing pictures and graphics. The JPEG process involves blocking, the discrete cosine transform, quantization, and lossless compression.
- MPEG (Moving Pictures Experts Group) compression is a method of compressing video. MPEG involves both spatial compression and temporal compression. The former is similar to JPEG, while the latter removes redundant frames.
- MP3 (MPEG audio layer 3) is a part of the MPEG standard. MP3 uses perceptual encoding techniques to compress CD-quality audio.

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. WCN 02-200-202

Copyright 2018 Cengage Learning. All Rights Reserved. May not be copied, scanned, or duplicated, in whole or in part. Due to electronic rights, some third party content may be suppressed from the eBook and/or eChapter(s). Editorial review has deemed that any suppressed content does not materially affect the overall learning experience. Cengage Learning reserves the right to remove additional content at any time if subsequent rights restrictions require it.